

Building and running PI emulator and assembler.

Prerequisites:

- g++ (version 12.3.1)
- cmake (version 3.27.7)
- C++ Standard Library

1. Getting the source code

```
git clone https://github.com/player400/pi.git
cd pi
```

```
[player402@fedora Pobrane]$ git clone https://github.com/player400/pi.git
Klonowanie do „pi”...
remote: Enumerating objects: 551, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 551 (delta 0), reused 1 (delta 0), pack-reused 545 (from 1)
Pobieranie obiektów: 100% (551/551), 378.97 KiB | 1.37 MiB/s, gotowe.
Rozwiązywanie delt: 100% (312/312), gotowe.
[player402@fedora Pobrane]$ cd pi
```

2. Building the emulator

```
cmake -S ./pi_vm -B emulator_build
cmake --build emulator_build
```

```
[player402@fedora pi]$ cmake -S ./pi_vm -B emulator_build
-- The C compiler identification is GNU 12.3.1
-- The CXX compiler identification is GNU 12.3.1
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Success
-- Found Threads: TRUE
-- Configuring done (0.3s)
-- Generating done (0.0s)
-- Build files have been written to: /home/player402/Pobrane/pi/emulator_build
[player402@fedora pi]$ cmake --build emulator_build
[ 50%] Building CXX object CMakeFiles/pi_vm.dir/main.cpp.o
[100%] Linking CXX executable pi_vm
[100%] Built target pi_vm
```

3. Building the assembler

```
cmake -S ./pi_asm -B assembler_build
cmake --build assembler_build
```

```
[player402@fedora pi]$ cmake -S ./pi_asm -B assembler_build
-- The C compiler identification is GNU 12.3.1
-- The CXX compiler identification is GNU 12.3.1
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done (0.3s)
-- Generating done (0.0s)
-- Build files have been written to: /home/player402/Pobrane/pi/assembler_build
[player402@fedora pi]$ cmake --build assembler_build
[ 3%] Building CXX object CMakeFiles/pi_asm.dir/main.cpp.o
[ 7%] Building CXX object CMakeFiles/pi_asm.dir/Tokens/Pattern.cpp.o
[11%] Building CXX object CMakeFiles/pi_asm.dir/Tokens/Instruction.cpp.o
[15%] Building CXX object CMakeFiles/pi_asm.dir/Tokens/Hook.cpp.o
[19%] Building CXX object CMakeFiles/pi_asm.dir/Tokens/Comment.cpp.o
[23%] Building CXX object CMakeFiles/pi_asm.dir/Tokens/MOV.cpp.o
[26%] Building CXX object CMakeFiles/pi_asm.dir/Tokens/FLAG.cpp.o
[30%] Building CXX object CMakeFiles/pi_asm.dir/Tokens/Value.cpp.o
[34%] Building CXX object CMakeFiles/pi_asm.dir/Tokens/Program.cpp.o
[38%] Building CXX object CMakeFiles/pi_asm.dir/Tokens/Expression.cpp.o
[42%] Building CXX object CMakeFiles/pi_asm.dir/Tokens/tokens.cpp.o
[46%] Building CXX object CMakeFiles/pi_asm.dir/Tokens/Logic.cpp.o
[50%] Building CXX object CMakeFiles/pi_asm.dir/Tokens/tokenizer.cpp.o
[53%] Building CXX object CMakeFiles/pi_asm.dir/Tokens/Variable.cpp.o
[57%] Building CXX object CMakeFiles/pi_asm.dir/Tokens/preprocessor.cpp.o
[61%] Building CXX object CMakeFiles/pi_asm.dir/Assembly/Entity.cpp.o
[65%] Building CXX object CMakeFiles/pi_asm.dir/Assembly/Assembly.cpp.o
/home/player402/Pobrane/pi/pi_asm/Assembly/Assembly.cpp: In member function 'int Assembly::getTrueValue(Value*)':
/home/player402/Pobrane/pi/pi_asm/Assembly/Assembly.cpp:296:1: warning: control reaches end of non-void function [-Wreturn-type]
 296 | }
     | ^
[69%] Building CXX object CMakeFiles/pi_asm.dir/Assembly/AssemblyInstruction.cpp.o
[73%] Building CXX object CMakeFiles/pi_asm.dir/Assembly/InstructionMOV.cpp.o
[76%] Building CXX object CMakeFiles/pi_asm.dir/Assembly/InstructionFLAG.cpp.o
[80%] Building CXX object CMakeFiles/pi_asm.dir/Assembly/Data.cpp.o
[84%] Building CXX object CMakeFiles/pi_asm.dir/Assembly/Writer/Writer.cpp.o
[88%] Building CXX object CMakeFiles/pi_asm.dir/Assembly/Writer/BinaryWriter.cpp.o
[92%] Building CXX object CMakeFiles/pi_asm.dir/Assembly/Writer/DecimalWriter.cpp.o
[96%] Building CXX object CMakeFiles/pi_asm.dir/Assembly/Writer/HexadecimalWriter.cpp.o
[100%] Linking CXX executable pi_asm
[100%] Built target pi_asm
```

4. Building and running a demo program

We will assemble and run an 8 bit Finite Impulse Response filter

```
./assembler_build/pi_asm -8 ./pi_asm/example_code/8_bit_only/soi8bit.pi -b example_exec.bin
```

Options explained:

- [-8|-4] – building either 4 bit or 8 bit program
- [-b|-d|-h] – either binary, decimal or hexadecimal output file

```
./emulator_build/pi_vm -b example_exec.bin -a 8
```

Options explained:

- [-b|-d|-h] – either binary, decimal or hexadecimal input file
- -a – architecture (8 bit (default) or 4 bit)

```
Output:      0      0x0  ''
Code execution STOPPED by an internal command. Enter a number and press enter to provide input. Type 'r' to resume e
xecution. Type 'e' to stop the emulator.
█
```

5. Using the emulator

A simple terminal user interface appears.

It enables you to:

- Use 8 bit parallel I/O port of the MCU
- Resume the execution of the program, after it has been stopped by setting the Half Flag to 1
- Exit the emulator

We will now compute the filter for values: 1 2 3

Type '1' and press enter:

```
Output:      1      0x1  ''
Code execution STOPPED by an internal command. Enter a number and press enter to provide input. Type 'r' to resume e
xecution. Type 'e' to stop the emulator.
█
```

The value has been saved into the Accumulator register (which is displayed as output in 3 formats: decimal, hexadecimal and as an ASCII character)

Type 'r' and press enter:

```
Output:      1      0x1  ''
Code execution STOPPED by an internal command. Enter a number and press enter to provide input. Type 'r' to resume e
xecution. Type 'e' to stop the emulator.
█
```

The program resumed and stopped again (to display output and prompt for more input). In this case, the output ('1') is equal to the input, so we don't see a change in value.

Now type in '2' and press enter, then type 'r' and press enter again:

```
Output:      3      0x3      ''
Code execution STOPPED by an internal command. Enter a number and press enter to provide input. Type 'r' to resume execution. Type 'e' to stop the emulator.
█
```

We see that next output of the filter is '3'

Now let's compute the next value (input '3'):

```
Output:      6      0x6      ''
Code execution STOPPED by an internal command. Enter a number and press enter to provide input. Type 'r' to resume execution. Type 'e' to stop the emulator.
█
```

Next output is '6'!

Now type 'e' to stop the emulator.

The emulator also has „print” mode. It is designed to print the output in a legible format instead of providing a terminal interface. Let's try it!

First, let's save our inputs into a file (remember to put r after each input, so that the MCU resumes its operation – otherwise you'll just quickly save 3 values into the Accumulator):

```
echo "1 r 2 r 3 r e" > input.txt
```

Then, let's use the emulator in print mode:

```
./emulator_build/pi_vm -b example_exec.bin -p < input.txt
```

Notice, that this time we skip the '-a 8', since it's redundant (8 is the default value)

```
[player402@fedora pi]$ ./emulator_build/pi_vm -b example_exec.bin -p < input.txt
0      0x0      ''
1      0x1      ''
3      0x3      ''
6      0x6      ''
█
```

We get the output!

We can also use the -c option, to count how many instructions were executed since the emulator was started:

```
[player402@fedora pi]$ ./emulator_build/pi_vm -b example_exec.bin -p -c < input.tx
0      0x0    ''      |      Instruction 7
1      0x1    ''      |      Instruction 503
3      0x3    ''      |      Instruction 1013
6      0x6    ''      |      Instruction 1537
```

Computing these 3 values took executing 1537 instructions!

We can also use -debug option, to run the debugger:

```
[player402@fedora pi]$ ./emulator_build/pi_vm -b example_exec.bin -debug
Debugger: Breakpoint hit at 16
Type: 'x <address>' to see value of a register or a memory address.
Type: 'b <address>' to add a breakpoint.
Type: 'f' to see all flags.
Type: 'n' to proceed to the next instruction.
Type: 'xr' to see values of all registers.
Type: 'c' to continue execution.
b 26
Breakpoint added!
c
Debugger: Breakpoint hit at 26
Type: 'x <address>' to see value of a register or a memory address.
Type: 'b <address>' to add a breakpoint.
Type: 'f' to see all flags.
Type: 'n' to proceed to the next instruction.
Type: 'xr' to see values of all registers.
Type: 'c' to continue execution.
n
Debugger: Stopped at 28
xr
Value of registry 0: 28 0x1c
Value of registry 1: 16 0x10
Value of registry 2: 163 0xa3
Value of registry 3: 0 0x0
Value of registry 4: 0 0x0
Value of registry 5: 0 0x0
Value of registry 6: 0 0x0
Value of registry 7: 0 0x0
Value of registry 8: 195 0xc3
Value of registry 9: 179 0xb3
Value of registry a: 240 0xf0
Value of registry b: 16 0x10
Value of registry c: 0 0x0
Value of registry d: 0 0x0
Value of registry e: 0 0x0
Value of registry f: 0 0x0
```

We can use breakpoints, stepping, as well as display values of memory and registers.

6. Assembly syntax

Here is a fragment of the example program we just ran (a function multiplying 2 8-bit unsigned integers):

```
//Function for multiplication of two 8-bit numbers. Arguments: R1, R2. Return Value: R3;
mul_ret: 0;
mul_iterator: 0;
mul_sum: 0;
mul_a: 0;
mul_minus1: 255;
mul_zero: 0;
mul:
    MOV 3, mul_zero;
    FLAG 1, 1;
        MOV 0, mul_ret;
    FLAG 1, 2;
        MOV 0, mul_ret;
    FLAG 1, false;
    MOV mul_sum, 3;
    MOV mul_a, 2;
    mul_ptl:
        MOV 2, mul_minus1;
        MOV mul_iterator, 3;
        MOV 1, mul_sum;
        MOV 2, mul_a;
        MOV mul_sum, 3;
        MOV 1, mul_iterator;
    FLAG 1, ~mul_iterator;
    MOV 0, mul_ptl_jump;
    FLAG 1, false;
    MOV 3, mul_sum;
MOV 0, mul_ret;
mul_func: mul;
mul_ptl_jump: mul_ptl;
```

As you know (if you don't – consult the ISA document) the PI assembly as 2 instructions: MOV and FLAG. In the assembly you can also use labels and data statements (for declaring data to be placed into the memory)

Syntax of the MOV instruction:

```
MOV <target>, <source>;
```

For example MOV 3, 192 will take the value from memory address 192 and put it in the register 3 (accumulator).

Syntax of the FLAG instruction:

```
FLAG <flag number>, <value>;
```

Flag number is a number from 0 to 3 (HF, SF, AF1, AF2), value is an arithmetical expression. Some examples:

```
FLAG 1, true; (set Skip Flag to true)
FLAG 3, false; (set AF2 to false)
FLAG 2, ~true; (set AF1 to false – negated true)
```

FLAG 0, (CARRY*45); (set HF to the logical product (AND) of carry bit from the last ALU operation and logical sum (OR) of all bits at memory address 45.

FLAG 1, ~(CARRY+35); (set SF to the negated logical sum (OR) of carry bit from the last ALU operation and logical sum (OR) of all bits at memory address 35.

Data statement:

```
<value> , <value2>;
```

For example:

```
213; (put the number 213 in this place in memory)
```

```
123, 56, 89; (put numbers 123, 56 and 89 in 3 consecutive bytes in memory)
```

You can also use **TIMES** keyword:

```
23, 49 TIMES 3; (this is equivalent to: 23, 49, 23, 49, 23, 49;)
```

Remember! Instructions need to be aligned to even addresses, so if you have data in-between instructions, make sure it's an even number of bytes.

Labels:

There are 2 ways to create a label.

The classic label:

```
my_label: (label will be equal to the address of instruction/data below it)
```

In this example, the instruction MOV 3, 2; will be skipped (by putting a value into the 0 register (which is the Program Counter) we are executing a jump)

```
MOV 0, ex_label;
MOV 3, 2;

ex_label:
MOV 1, 0;
```

And label in an instruction:

```
MOV 4, >label;
MOV label, 9;
FLAG 2, ~(>label);
```

This kind of label will point at the second byte of an instruction word.

For example:

`MOV LBL, 3;` (pasting the contents of register 3 (accumulator) into the address field of the next instruction)

`MOV 2, >LBL;` (placing in register 2 the value from the address given by the previous instruction)

If the value '45' was in the accumulator (register 3), then contents of memory address 45 will be put in register 2. That's because label 'LBL' points to the address, where second byte of the instruction is.

This way you can use pointers.

Comments:

```
//<comment> ;
```

You start comments with // and end them with ;

Example:

```
program_start;
0 TIMES 3;
0 TIMES 4;
delay_ptr_minus: delay;
coeff_ptr_minus: coeff;
minusL: 240;
L: 16; //Delay line length;
j: 0; //Inner loop/coeff iterator;
sum: 0;
//Delay line iterators;
delay_begin: 0;
delay_ind: 0;
```

```
program_start:

//delay_ptr_minus and coeff_ptr_minus are supposed to contain delay_ptr+L and coeff_ptr+L,
MOV 1, L;
MOV 2, delay_ptr_minus;
MOV delay_ptr_minus, 3;
MOV 2, coeff_ptr_minus;
```

You can see here, that label `program_start` points to the beginning of the program.

It is then used in the data statement in the first line of the code.

This way it will be placed at memory address 0 (which is also register 0, so the Program Counter).

Next you can see `0 TIMES 3;` - that means 0s will be placed at addresses 1, 2 and 3 (so also registers 1, 2 and 3).